

pandas & polars

Lecture 08

Dr. Colin Rundel

NumPy dtypes

dtype

Every NumPy array stores elements of a single type, its `dtype`. As with R's atomic vectors the type is inferred at creation and mixed values are coerced, but NumPy has many more types and each has an explicit size - `int8` to `int64`, `uint8` to `uint64`, `float16` to `float64`, `bool`, `complex128`, etc.

```
1 np.array([1, 2, 3]).dtype
```

py

```
dtype('int64')
```

```
1 np.array([1.5, 2]).dtype
```

py

```
dtype('float64')
```

```
1 np.array([True, False]).dtype
```

py

```
dtype('bool')
```

```
1 np.array(["a", "bcd"]).dtype
```

py

```
dtype('<U3')
```

```
1 np.array([1, 2.5, True]).dtype
```

py

```
dtype('float64')
```

```
1 np.array([1, 2, 3], dtype="float32")
```

py

```
array([1., 2., 3.], dtype=float32)
```

```
1 np.array([1.7, 2.2]).astype("int64")
```

py

```
array([1, 2])
```

dtypes are fixed

The dtype is set when the array is created and does not change. Where an R vector is promoted to fit what is assigned into it, a NumPy array converts the value to its own dtype.

```
1 x = c(1L, 2L, 3L)
2 typeof(x)
```

R

```
[1] "integer"
```

```
1 x[1] = 2.9
2 x
```

R

```
[1] 2.9 2.0 3.0
```

```
1 typeof(x)
```

R

```
[1] "double"
```

```
1 x = np.array([1, 2, 3])
2 x.dtype
```

py

```
dtype('int64')
```

```
1 x[0] = 2.9
2 x
```

py

```
array([2, 2, 3])
```

```
1 y = np.array([100, 200], dtype="uint8")
2 y + y
```

py

```
array([200, 144], dtype=uint8)
```

pandas

pandas

Python has no built-in tabular data structure. pandas (2008) is an established choice - it grew out of NumPy and borrows heavily from R. Today we will focus on some of the key differences between the two implementations.

```
1 import pandas as pd
2 pd.__version__
```

py

'3.0.5'

Three classes do the heavy lifting:

- **Series** - a 1d typed array with an index (a column)
- **Index** - the labels attached to rows and columns
- **DataFrame** - a collection of **Series** sharing an index (a table)

pandas 3.0 (released early 2026) changed some long standing defaults - strings have their own dtype and copy-on-write

Sta 523 - Fall 2026

Series

A *Series* is a 1d array with a dtype together with an *index* of labels - the closest thing in R is a named vector. The difference is that a Series always has an index (integer positions by default) and the index participates in lookup, alignment, and joins rather than being decoration/annotation.

```
1 s = pd.Series([4, 2, 1, 3])
```

py

```
1 s
```

py

```
0    4
1    2
2    1
3    3
dtype: int64
```

```
1 s.index
```

py

```
RangeIndex(start=0, stop=4, step=1)
```

```
1 pd.Series(
2     [4, 2, 1, 3],
3     index=["a", "b", "c", "d"]
4 )
```

py

```
a    4
b    2
c    1
d    3
dtype: int64
```

```
1 pd.Series({"a": 4, "b": 2})
```

py

```
a    4
b    2
dtype: int64
```

dtypes

pandas dtypes come from NumPy (`int64`, `float64`, `bool`, `datetime64`) plus extension dtypes (`str`, `category`, `Int64`, ...).

For mixtures such as numbers and strings, pandas falls back to `object` rather than coercing everything to strings as R does. An object Series still supports elementwise operations, but storing arbitrary Python objects usually costs more memory and computation.

```
1 pd.Series([1, 2, 3]).dtype
```

py

```
dtype('int64')
```

```
1 pd.Series([1.5, 2]).dtype
```

py

```
dtype('float64')
```

```
1 pd.Series([True, False]).dtype
```

py

```
dtype('bool')
```

```
1 pd.Series(["a", "b"]).dtype
```

py

```
<StringDtype(na_value=nan)>
```

```
1 pd.Series([1, "a", True]).dtype
```

py

```
dtype('O')
```

```
1 pd.Series([1, "a", True])
```

py

```
0      1
1      a
2     True
dtype: object
```

```
1 pd.Series(["1", "2"]).astype("int64")
```

py

```
0      1
1      2
dtype: int64
```

Labels vs positions

A pandas index can hold integers, strings, etc. A scalar `s[key]` looks up a label, even when the key is an integer. Use `.iloc` for positions and `.loc` for labels.

```
1 t = pd.Series({"a": 4, "b": 2, "c": 1, "d": 3})
2 u = pd.Series([4, 2, 1, 3], index=[3, 2, 1, 0])
```

```
1 t["b"]
```

py

np.int64(2)

```
1 t[1]
```

py

KeyError: 1

```
1 t.iloc[1]
```

py

np.int64(2)

```
1 u["b"]
```

py

KeyError: 'b'

```
1 u[1]
```

py

np.int64(1)

```
1 u.iloc[1]
```

py

np.int64(2)

Slices: labels vs positions

`.loc` label slices include both endpoints; `.iloc` position slices exclude the stop. Integer slices in `s[...]` are positional, unlike scalar integer lookups.

```
1 t.loc["b":"c"]
```

py

```
b    2
c    1
dtype: int64
```

```
1 t.iloc[1:3]
```

py

```
b    2
c    1
dtype: int64
```

```
1 u[1:3]
```

py

```
2    2
1    1
dtype: int64
```

```
1 u.loc[3:1]
```

py

```
3    4
2    2
1    1
dtype: int64
```

Index alignment

Arithmetic between Series aligns on index rather than position. The result spans the union of both indexes with **NaN** for labels missing on one side. Assigning a Series to a column in a DataFrame aligns the same way.

```
1 m = pd.Series([1, 2, 3], index=["a", "b", "c"])
2 n = pd.Series([10, 20, 30], index=["c", "b", "a"])
3 d = pd.DataFrame({"x": [1, 2, 3]}, index=["a", "b", "c"])
```

```
1 m + n
```

```
a    31
b    22
c    13
dtype: int64
```

```
1 m + n[["a", "c"]]
```

```
a    31.0
b     NaN
c    13.0
dtype: float64
```

```
1 d["y"] = n[["a", "c"]]
2 d
```

	x	y
a	1	30.0
b	2	NaN
c	3	10.0

DataFrame

A `DataFrame` is a dict-like collection of Series sharing a row index, (keys are the column indexes).

```
1 penguins = pd.read_csv("data/penguins.csv"); penguins
```

	species	island	bill_length_mm	...	body_mass_g	sex	year
0	Adelie	Torgersen	39.1	...	3750.0	male	2007
1	Adelie	Torgersen	39.5	...	3800.0	female	2007
..	...	...	...	...	...	...	...
342	Chinstrap	Dream	50.8	...	4100.0	male	2009
343	Chinstrap	Dream	50.2	...	3775.0	female	2009

[344 rows x 8 columns]

```
1 penguins.index
```

RangeIndex(start=0, stop=344, step=1)

```
1 penguins.columns
```

```
Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',  
      'flipper_length_mm', 'body_mass_g', 'sex', 'year'],  
      dtype='str')
```

The primary difference with R data frames is automatic alignment on the row index. R's `row.names` support printing and

Column types - csv

Types are guessed from the file - there is no factor type by default (`category` is opt in) and the two integer columns with missing values were upcast to `float64`. Using the `numpy_nullable` backend uses pandas' extension dtypes instead, which keep integers as integers (more on these shortly).

```
1 penguins.dtypes
```

py

```
species          str
island           str
bill_length_mm   float64
bill_depth_mm    float64
flipper_length_mm float64
body_mass_g      float64
sex              str
year             int64
dtype: object
```

```
1 pd.read_csv(
2     "data/penguins.csv",
3     dtype_backend="numpy_nullable"
4 ).dtypes
```

py

```
species          string
island           string
bill_length_mm    Float64
bill_depth_mm     Float64
flipper_length_mm Int64
body_mass_g       Int64
sex              string
year             Int64
dtype: object
```

Missing values

R has typed **NA** values for logical, integer, double, complex, and character vectors. NumPy has no universal missing-value sentinel across dtypes: floating point arrays can use **NaN** and datetimes can use **NaT**, but integers and booleans have no equivalent. With pandas' default inference, missing integers become **float64**, while the boolean example below becomes **object** holding **None**.

```
1 pd.Series([1, 2, None])
```

```
0    1.0
1    2.0
2    NaN
dtype: float64
```

```
1 pd.Series([True, False, None])
```

```
0    True
1   False
2    None
dtype: object
```

```
1 pd.Series(["a", None])
```

```
0    a
1   NaN
dtype: str
```

```
1 pd.Series(
2     [pd.Timestamp("2026-09-15"), None]
3 )
```

```
0    2026-09-15
1              NaT
dtype: datetime64[us]
```

Detecting missing values

`NaN` is not equal to anything, including itself, so `==` cannot find missing values - `isna()` (and `notna()`) recognize `NaN`, `None`, `NaT`, and `pd.NA` alike. `dropna()` and `fillna()` are the usual next steps.

```
1 s = pd.Series([1, 2, None])
2 s == np.nan
```

py

```
0    False
1    False
2    False
dtype: bool
```

```
1 s.isna()
```

py

```
0    False
1    False
2     True
dtype: bool
```

```
1 s.isna().sum()
```

py

```
np.int64(1)
```

```
1 s.dropna()
```

py

```
0    1.0
1    2.0
dtype: float64
```

```
1 s.fillna(0)
```

py

```
0    1.0
1    2.0
2    0.0
dtype: float64
```

Missing values in aggregations

Reductions skip missing values by default (`skipna=True`), the opposite of R's `na.rm = FALSE`.

```
1 mass = penguins["body_mass_g"]
```

py

```
1 mass.mean()
```

py

```
np.float64(4201.754385964912)
```

```
1 mass.mean(skipna=False)
```

py

```
np.float64(nan)
```

```
1 mass.count()
```

py

```
np.int64(342)
```

```
1 penguins.dropna().shape
```

py

```
(333, 8)
```

```
1 penguins.isna().sum()
```

py

```
species           0
island            0
bill_length_mm    2
bill_depth_mm     2
flipper_length_mm 2
body_mass_g       2
sex              11
year             0
dtype: int64
```

Nullable dtypes

The extension dtypes `Int64`, `Float64`, `boolean`, and `string` use `pd.NA` to represent missing values without changing dtype. Only the numeric names here are capitalized. These types are opt in, via `dtype=`, `dtype_backend=` when reading, or `convert_dtypes()`. Arithmetic and comparisons use three valued logic - `pd.NA == 1` is `<NA>` rather than `False`.

```
1 t = pd.Series([1, 2, None], dtype="Int64")
```

py

```
1 t
```

py

```
0      1
1      2
2    <NA>
dtype: Int64
```

```
1 t > 1
```

py

```
0    False
1     True
2    <NA>
dtype: boolean
```

```
1 pd.NA == 1
```

py

```
<NA>
```

```
1 pd.NA | True
```

py

```
True
```

```
1 pd.NA & True
```

py

```
<NA>
```

```
1 bool(pd.NA)
```

py

```
TypeError: boolean value of NA is ambiguous
```

So a pandas column can hold `NaN`, `None`, `NaT`, or `pd.NA` depending on its dtype - `isna()` treats all of them as missing,

Subsetting with []

`df[...]` is overloaded on the type of its argument - a string picks a column, a list of strings a set of columns, and a slice or boolean Series picks rows. There is no two argument `df[i, j]` form.

```
1 penguins["species"]
```

py

```
0      Adelie
1      Adelie
...
342  Chinstrap
343  Chinstrap
Name: species, Length: 344, dtype: str
```

```
1 penguins[["species", "island"]]
```

py

```
      species  island
0      Adelie  Torgersen
1      Adelie  Torgersen
..         ...      ...
342  Chinstrap    Dream
343  Chinstrap    Dream
```

[344 rows x 2 columns]

```
1 penguins[penguins["body_mass_g"] > 6000]
```

py

```
   species  island  bill_length_mm  ...  body_mass_g  sex  year
169  Gentoo  Biscoe           49.2  ...     6300.0  male  2007
185  Gentoo  Biscoe           59.6  ...     6050.0  male  2007
```

[2 rows x 8 columns]

.loc and .iloc

Two dimensional subsetting like R's `df[i, j]` goes through `.loc` (labels) and `.iloc` (positions), each taking `[rows, cols]`. Label slices include the stop; position slices exclude it. Both accept boolean arrays, but an indexed boolean Series belongs with `.loc`, not `.iloc`.

```
1 penguins.loc[0:2, "species":"island"]
```

py

	species	island
0	Adelie	Torgersen
1	Adelie	Torgersen
2	Adelie	Torgersen

```
1 penguins.iloc[0:2, 0:2]
```

py

	species	island
0	Adelie	Torgersen
1	Adelie	Torgersen

```
1 penguins.loc[penguins["species"] == "Gentoo", ["species", "body_mass_g"]]
```

py

	species	body_mass_g
152	Gentoo	4500.0
153	Gentoo	5700.0
...	...	...
274	Gentoo	5200.0
275	Gentoo	5400.0

[124 rows x 2 columns]

Combining conditions

Use `&` for elementwise AND, `|` for OR, and `~` for NOT. Parenthesize comparisons; Python's `and` / `or` do not combine Series. `.isin()` tests membership in a collection.

```
1 mask = (  
2     penguins["species"].isin(["Adelie", "Gentoo"])  
3     & (penguins["body_mass_g"] > 5000)  
4 )  
5 penguins.loc[mask, ["species", "body_mass_g"]].head(3)
```

	species	body_mass_g
153	Gentoo	5700.0
155	Gentoo	5700.0
156	Gentoo	5400.0

```
1 penguins.loc[  
2     (penguins["species"] == "Gentoo")  
3     | (penguins["body_mass_g"] > 5000),  
4     ["species", "body_mass_g"]  
5 ].head(3)
```

	species	body_mass_g
152	Gentoo	4500.0
153	Gentoo	5700.0
154	Gentoo	4450.0

Rows are Series too

Selecting a single row returns a Series, and a Series has one dtype - so a heterogeneous row is upcast to `object`. R keeps a single row as a data frame for exactly this reason.

```
1 cols = ["species", "year"]
```

```
1 penguins.loc[0, cols]
```

```
species    Adelie
year        2007
Name: 0, dtype: object
```

```
1 r = penguins.loc[[0], cols]
2 r
```

```
species  year
0 Adelie  2007
```

```
1 r.dtypes
```

```
species    str
year       int64
dtype: object
```

Mutation

DataFrames are mutable - `df["z"] = ...` changes the object in place, and any other name bound to the same object sees the change (reference semantics). This is unlike R's copy-on-modify, where the same code would leave `d` untouched. `copy()` makes an independent data frame, and methods such as `assign()` return a new one.

```
1 d = pd.DataFrame({"x": [1, 2, 3]})
2 d2 = d
3 d2["y"] = d2["x"] * 2
4 d
```

	x	y
0	1	2
1	2	4
2	3	6

```
1 d3 = d.copy()
2 d3["z"] = 1
3 d
```

	x	y
0	1	2
1	2	4
2	3	6

pandas 3.0 enforces copy-on-write: assigning through a subset does not update its source. Binding two names to the same DataFrame still creates an alias. This protection does not recursively copy mutable Python objects stored inside

Updating selected rows

Use a single `.loc[rows, column] = value` assignment on the frame you want to change. Chained assignment such as `df["x"][mask] = value` cannot update the original frame under copy-on-write.

```
1 updated = penguins.copy()
2 updated["size"] = "other"
3 updated.loc[updated["body_mass_g"] > 5000, "size"] = "large"
4 updated.loc[updated["size"] == "large", ["body_mass_g", "size"]].head(3)
```

	body_mass_g	size
153	5700.0	large
155	5700.0	large
156	5400.0	large

Referring to columns

dplyr's `filter(df, x > 1)` works because R captures the unevaluated expression. Python evaluates arguments eagerly and a bare `x` is a `NameError`, so pandas offers several workarounds, none of which are quite as convenient.

```
1 pg = penguins[["species", "island", "body_mass_g"]]
```

py

Explicit - reuse the object,

```
1 pg[pg["body_mass_g"] > 6000]
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

String - DSL parsed by pandas,

```
1 pg.query("body_mass_g > 6000")
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

Callable - a function / lambda

```
1 pg[lambda d: d["body_mass_g"] > 6000]
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

Expression - `pd.col()`,

```
1 pg[pd.col("body_mass_g") > 6000]
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

Method chaining

There is no pipe operator, but since nearly every method returns a new DataFrame the equivalent is method chaining. Callables and `pd.col()` are what let a step refer to columns of the intermediate result. Subsetting with `[]` can be a step in the chain like any other.

```
1 (penguins
2   .query("species == 'Gentoo'")
3   .assign(kg = pd.col("body_mass_g") / 1000)
4   .sort_values("kg", ascending=False)
5   [["species", "island", "kg"]])
6   .head(3)
7 )
```

	species	island	kg
169	Gentoo	Biscoe	6.30
185	Gentoo	Biscoe	6.05
229	Gentoo	Biscoe	6.00

groupby

Split-apply-combine works the same way - `groupby()` returns a `DataFrameGroupBy` which is then used by subsequent aggregation calls. By default the group keys become the *index* of the result (`as_index=False` keeps them as columns, like `.by`). Named aggregations use `(column, function)` pairs.

```
1 g = penguins.groupby("species")
2 type(g)
```

py

```
<class 'pandas.api.typing.DataFrameGroupBy'>
```

```
1 g["body_mass_g"].mean()
```

py

```
species
Adelie      3700.662252
Chinstrap   3733.088235
Gentoo      5076.016260
Name: body_mass_g, dtype: float64
```

```
1 (penguins
2   .groupby("species", as_index=False)
3   .agg(
4       n = ("species", "size"),
5       mass = ("body_mass_g", "mean")
6   )
7 )
```

py

	species	n	mass
0	Adelie	152	3700.662252
1	Chinstrap	68	3733.088235
2	Gentoo	124	5076.016260

Grouped transforms

`transform()` returns a result the same length as its input, broadcasting each group's value back over that group's rows - the equivalent of `mutate()` with `.by`. Since the result shares the frame's index it can be used directly in arithmetic or assignment.

```
1 avg = g["body_mass_g"].transform("mean")
2 avg.head(3)
```

py

```
0    3700.662252
1    3700.662252
2    3700.662252
Name: body_mass_g, dtype: float64
```

```
1 penguins.assign(rel = pd.col("body_mass_g") / avg)[["species", "rel"]].head(3)
```

py

```
   species      rel
0  Adelie  1.013332
1  Adelie  1.026843
2  Adelie  0.878221
```

MultiIndex

Group by two keys and the result has a hierarchical row `MultiIndex`; aggregate a column two ways with the dictionary/list syntax and the columns get one too. The tidyverse usually keeps group keys and summaries as ordinary columns.

```
1 (penguins
2   .groupby(["species", "island"])
3   ["body_mass_g"].mean()
4   )
```

species	island	
Adelie	Biscoe	3709.659091
	Dream	3688.392857
	Torgersen	3706.372549
Chinstrap	Dream	3733.088235
Gentoo	Biscoe	5076.016260

Name: body_mass_g, dtype: float64

```
1 (penguins
2   .groupby("species")
3   .agg({
4       "body_mass_g": ["mean", "max"]
5   })
6   )
```

	body_mass_g	
	mean	max
species		
Adelie	3700.662252	4775.0
Chinstrap	3733.088235	4800.0
Gentoo	5076.016260	6300.0

`reset_index()` moves row index levels into columns, but does not flatten hierarchical columns. Use `as_index=False`

Exercise 1

Using pandas and `data/flights.parquet`, answer the following. Questions 1 and 2 are the core; 3 and 4 are extensions if time allows.

1. How many flights to LAX did each legacy carrier (AA, UA, DL, US) have in May from JFK, and what was their mean actual air time (`air_time`)? Report only carriers with matching records.
2. Which plane (`tailnum`) has the most flight records from each New York airport? Exclude missing tail numbers and return all ties.
3. Which five calendar dates (`year`, `month`, `day`) had the lowest mean departure delay? Skip missing delays and break ties by earliest date.
4. Which flight has the largest arrival delay as a percentage of its actual air time ($100 * \text{arr_delay} / \text{air_time}$)? Exclude missing values and nonpositive air times, and return all ties.



polars

polars (2020) is a newer data frame library written in Rust on top of Apache Arrow's memory layout, with bindings for Python, R, and JavaScript. Rather than extend NumPy conventions it was designed from scratch around a few ideas,

- No index - rows are only ever positions
- A strict schema - columns have explicit types and incompatible values are rejected by default; expressions can promote types or replace columns
- Expressions - computations on columns are objects, built up and then evaluated in a context
- Lazy evaluation - queries are planned and optimized before running, on all available cores

```
1 import polars as pl
2 pl.__version__
```

py

```
'1.44.2'
```

Series

A polars Series is a name, a dtype, and values - nothing else. With no index there is nothing to align on, so arithmetic is positional and lengths must match (or one side has length one). This is R's rule without recycling.

```
1 pl.Series("x", [4, 2, 1, 3])
```

py

shape: (4,)

x
4
2
1
3

```
1 a = pl.Series([1, 2, 3])
2 b = pl.Series([10, 20, 30])
3 a + b
```

py

shape: (3,)

11
22
33

DataFrame

The print shows the shape and every column's dtype, much like a tibble. Note that the integer columns stayed integers despite the missing values, and that "NA" had to be declared as a missing value marker - polars does not guess.

```
1 penguins = pl.read_csv("data/penguins.csv", null_values="NA"); penguins
```

shape: (344, 8)

species	island	bill_length_mm	...	body_mass_g	sex	year
str	str	f64	...	i64	str	i64
"Adelie"	"Torgersen"	39.1	...	3750	"male"	2007
"Adelie"	"Torgersen"	39.5	...	3800	"female"	2007
"Adelie"	"Torgersen"	40.3	...	3250	"female"	2007
...	...	...	...	...	...	...
"Chinstrap"	"Dream"	49.6	...	3775	"male"	2009
"Chinstrap"	"Dream"	50.8	...	4100	"male"	2009
"Chinstrap"	"Dream"	50.2	...	3775	"female"	2009

Subsetting

Without an index there is no `.loc` / `.iloc` distinction - `df[...]` takes column names or row positions and a slice is just a slice. This is fine for a quick look, but the idiomatic way to subset is with expressions in `filter()` and `select()`.

```
1 penguins["species"]
```

py

shape: (344,)

species
str
"Adelie"
"Adelie"
"Adelie"
...
"Chinstrap"
"Chinstrap"
"Chinstrap"

```
1 penguins[0:3, ["species", "island"]]
```

py

shape: (3, 2)

species	island
str	str
"Adelie"	"Torgersen"
"Adelie"	"Torgersen"
"Adelie"	"Torgersen"

```
1 penguins[0, "species"]
```

py

'Adelie'

Missing values

Missing data is `null` in every dtype (like R's `NA`) and `NaN` is a floating point value, distinct from missing. `mean()` and `sum()` skip `null` by default and propagate `NaN`. Ordinary `min()` and `max()` ignore `NaN`; `nan_min()` and `nan_max()` propagate it.

```
1 s = pl.Series([1.0, None, np.nan])
```

py

```
1 s
```

py

shape: (3,)

f64
1.0
null
NaN

```
1 s+1
```

py

shape: (3,)

f64
2.0
null
NaN

```
1 s.is_null().to_list()
```

py

[False, True, False]

```
1 s.is_nan().to_list()
```

py

[False, None, True]

```
1 s.mean()
```

py

nan

```
1 s.fill_nan(None).mean()
```

py

1.0

Expressions

`pl.col("x")` does not return a column, it returns an *expression* - a description of a computation that is not attached to any DataFrame and has not been evaluated. Expressions compose with operators and methods and are only evaluated when handed to a context. This is polars' answer to data masking - build an object that represents the code rather than capturing it.

```
1 mass_kg = pl.col("body_mass_g") / 1000
2 mass_kg
```

```
[(col("body_mass_g")) / (dyn int: 1000)]
```

```
1 type(mass_kg)
```

```
<class 'polars.expr.expr.Expr'>
```

```
1 pl.col("body_mass_g").mean().alias("avg_mass")
```

```
col("body_mass_g").mean().alias("avg_mass")
```

Contexts

A context is a DataFrame method that evaluates expressions against its columns, each corresponding to a dplyr verb,

Context	Result	dplyr
<code>select()</code>	a new frame containing only these expressions	<code>select(), summarize()</code>
<code>with_columns()</code>	the frame with these expressions added	<code>mutate()</code>
<code>filter()</code>	the rows where these expressions are true	<code>filter()</code>
<code>group_by().agg()</code>	one row per group of these expressions	<code>group_by() > summarize()</code>

```
1 penguins.select(  
2   "species", mass_kg.alias("mass_kg")  
3 ).head(2)
```

py

shape: (2, 2)

species	mass_kg
str	f64
"Adelie"	3.75
"Adelie"	3.8

```
1 penguins.filter(  
2   pl.col("body_mass_g") > 6000  
3 ).select("species", "body_mass_g")
```

py

shape: (2, 2)

species	body_mass_g
str	i64
"Gentoo"	6300
"Gentoo"	6050

Combining predicates

Multiple predicates in `filter()` are combined with AND. Use `.is_in()` for membership and `|` for OR; parenthesize comparisons when combining them with operators.

```
1 penguins.filter(  
2   pl.col("species").is_in(["Adelie", "Gentoo"]),  
3   pl.col("body_mass_g") > 5000  
4 ).select("species", "body_mass_g").head(3)
```

shape: (3, 2)

species	body_mass_g
str	i64
"Gentoo"	5700
"Gentoo"	5700
"Gentoo"	5400

```
1 penguins.filter(  
2   (pl.col("species") == "Gentoo")  
3   | (pl.col("body_mass_g") > 5000)  
4 ).select("species", "body_mass_g").head(3)
```

shape: (3, 2)

Expression expansion

One expression can expand to many columns - by listing names, by dtype, or with selectors from `polars.selectors`. This is `across()` with tidy selection, available in every context.

```
1 import polars.selectors as cs
```

py

```
1 penguins.select(cs.ends_with("_mm").mean().name.suffix("_mean"))
```

py

shape: (1, 3)

bill_length_mm_mean	bill_depth_mm_mean	flipper_length_mm_mean
f64	f64	f64
43.92193	17.15117	200.915205

```
1 penguins.select(pl.col(pl.String).n_unique())
```

py

shape: (1, 3)

species	island	sex
u32	u32	u32
3	3	3

group_by

`group_by()` followed by `agg()` evaluates each expression once per group and the keys come back as ordinary columns (like `.by`), in no guaranteed order. Since expressions are values the same one can be reused across contexts, and `.over()` turns an aggregation into a window computation for a grouped mutate.

```
1 mass = pl.col("body_mass_g")
2 penguins.group_by("species").agg(
3     n = pl.len(), avg = mass.mean()
4 ).sort("species")
```

py

shape: (3, 3)

species	n	avg
str	u32	f64
"Adelie"	152	3700.662252
"Chinstrap"	68	3733.088235
"Gentoo"	124	5076.01626

```
1 avg = mass.mean().over("species")
2 (penguins
3     .with_columns(rel = mass / avg)
4     .select("species", "rel")
5     .head(2)
6 )
```

py

shape: (2, 2)

species	rel
str	f64
"Adelie"	1.013332
"Adelie"	1.026843

Lazy evaluation

A `LazyFrame` shares the expression-based query API of a `DataFrame` but records transformations instead of running them. The `scan_*()` readers start lazy and `.lazy()` converts an existing frame. Planning may read metadata; `.collect()` executes the query and materializes its result.

```
1 q = (  
2     pl.scan_parquet("data/flights.parquet")  
3     .filter(pl.col("dest") == "LAX")  
4     .group_by("carrier")  
5     .agg(pl.len(), pl.col("dep_delay").mean())  
6     .sort("len", descending=True)  
7 )  
8 type(q)
```

```
<class 'polars.lazyframe.frame.LazyFrame'>
```

Recording the whole query before running it is what makes optimization possible - the same idea underlies SQL databases and dbplyr, which we will see later in the course.

Query plan

Before running, the plan is optimized - here only 3 of the 19 columns are read from the parquet file and the filter is applied during the scan (projection and predicate *pushdown*).

```
1 print( q.explain() )
```

py

```
SORT BY [descending: [true]] [col("len")]
  AGGREGATE[maintain_order: false]
    [len(), col("dep_delay").mean()] BY [col("carrier")]
  FROM
    simple π 2/2 ["carrier", "dep_delay"]
      Parquet SCAN [data/flights.parquet]
      PROJECT 3/19 COLUMNS
      SELECTION: col("dest") == "LAX"
      ESTIMATED ROWS: 336776
```

Collecting

`collect()` runs the optimized plan and returns an ordinary DataFrame. Since the plan is known up front it can also be executed in a streaming fashion, in batches, for data larger than memory (`collect(engine="streaming")`).

```
1 q.collect()
```

py

shape: (5, 3)

carrier	len	dep_delay
str	u32	f64
"UA"	5823	10.757738
"AA"	3582	9.168119
"VX"	2580	10.56189
"DL"	2501	5.668539
"B6"	1688	8.994652

Speed - reading CSV

Both libraries use compiled code. Polars can use multiple threads and optimize lazy queries, but timings depend on the data and workload. Start by timing the same CSV read, including parsing and type inference.

```
1 import timeit
2 def time(f): return round( min(timeit.repeat(f, number=1, repeat=5)), 4 )
```

```
1 time(lambda: pd.read_csv("data/flights.csv"))
```

0.2204

```
1 time(lambda: pl.read_csv("data/flights.csv", null_values="NA"))
```

0.01

Timings are seconds: the minimum of five runs on this machine. Repeated reads may use the operating system's file

Speed - data already in memory

Both frames are loaded before timing. Each computation groups all flights by carrier, calculates the mean departure delay, and sorts by carrier.

```
1 flights_pd = pd.read_parquet("data/flights.parquet")
2 flights_pl = pl.read_parquet("data/flights.parquet")
3
4 time(lambda: (
5     flights_pd.groupby("carrier", as_index=False)["dep_delay"]
6     .mean().sort_values("carrier")
7 ))
```

py

0.003

```
1 time(lambda: (
2     flights_pl.group_by("carrier").agg(pl.col("dep_delay").mean())
3     .sort("carrier")
4 ))
```

py

0.0015

Speed - from file to result

Now include Parquet reading. Both queries filter to LAX, count flights and average departure delay by carrier, then sort by count. Polars uses the lazy query `q` and can push projection and filtering into its scan.

```
1 def lax_pandas():
2     return (
3         pd.read_parquet(
4             "data/flights.parquet",
5             columns=["dest", "carrier", "dep_delay"]
6         )
7         .query("dest == 'LAX'")
8         .groupby("carrier", as_index=False)
9         .agg(len=("carrier", "size"), dep_delay=("dep_delay", "mean"))
10        .sort_values("len", ascending=False)
11    )
12
13 time(lax_pandas)
```

0.0081

```
1 time(lambda: q.collect())
```

0.0015

Interop

Arrow supports exchange between R, pandas, and polars; reticulate also converts between R and Python objects. Conversion can change dtypes and missing-value representation. For files, parquet is a shared format (Lecture 07).

Default `to_pandas()` converts nullable integer columns here to floats. Arrow extension arrays preserve their integer types and nulls.

```
1 penguins.to_pandas().dtypes
```

py

```
species          str
island           str
bill_length_mm   float64
bill_depth_mm    float64
flipper_length_mm float64
body_mass_g      float64
sex              str
year             int64
dtype: object
```

```
1 penguins.to_pandas(
2     use_pyarrow_extension_array=True
3 ).dtypes
```

py

```
species          large_string[pyarrow]
island           large_string[pyarrow]
bill_length_mm   double[pyarrow]
bill_depth_mm    double[pyarrow]
flipper_length_mm int64[pyarrow]
body_mass_g      int64[pyarrow]
sex              large_string[pyarrow]
year             int64[pyarrow]
dtype: object
```

Exercise 2

Using polars and `data/flights.parquet`, repeat the questions from Exercise 1. Questions 1 and 2 are the core; 3 to 5 are extensions if time allows.

1. How many flights to LAX did each legacy carrier (AA, UA, DL, US) have in May from JFK, and what was their mean actual air time (`air_time`)? Report only carriers with matching records.
2. Which plane (`tailnum`) has the most flight records from each New York airport? Exclude missing tail numbers and return all ties.
3. Which five calendar dates (`year`, `month`, `day`) had the lowest mean departure delay? Skip missing delays and break ties by earliest date.
4. Which flight has the largest arrival delay as a percentage of its actual air time ($100 * \text{arr_delay} / \text{air_time}$)? Exclude missing values and nonpositive air times, and return all ties.
5. Rewrite question 1 to start from `pl.scan_parquet()` and look at `explain()` before collecting. How many of the 19 columns does polars read?

Summary

Three data frames

Concept	R / tibble	pandas	polars
column	atomic vector	<code>Series</code> (values + index)	<code>Series</code> (values only)
row labels	<code>row.names</code> , rarely used	<code>Index</code> , central	none
alignment	by position, with recycling	by index label	by position, lengths must match
mixed types	coerced to a common type	<code>object</code> dtype	error
missing values	<code>NA</code> in any type	<code>NaN</code> / <code>None</code> / <code>NaT</code> / <code>pd.NA</code> by dtype	<code>null</code> in any type
column references	data masking	strings, callables, <code>pd.col()</code>	<code>pl.col()</code> expressions
mutation	copy-on-modify	in place, copy-on-write for subsets	methods return new frames
grouped result	keys as columns	keys as (Multi)Index	keys as columns
laziness	none (dbplyr, dtplyr)	none	<code>LazyFrame</code> + optimizer
engine	C, single threaded	NumPy, single threaded	Rust + Arrow, multithreaded

Verbs

dplyr	pandas	polars
<code>filter(x > 1)</code>	<code>query("x > 1"), [pd.col("x") > 1]</code>	<code>filter(pl.col("x") > 1)</code>
<code>select(x, y)</code>	<code>[["x", "y"]], filter(regex=)</code>	<code>select("x", "y"), select(cs....())</code>
<code>mutate(z = x * 2)</code>	<code>assign(z = pd.col("x") * 2)</code>	<code>with_columns(z = pl.col("x") * 2)</code>
<code>arrange(desc(x))</code>	<code>sort_values("x", ascending=False)</code>	<code>sort("x", descending=True)</code>
<code>summarize(m = mean(x))</code>	<code>agg(m = ("x", "mean"))</code>	<code>select(m = pl.col("x").mean())</code>
<code>group_by(g)</code>	<code>groupby("g", as_index=False)</code>	<code>group_by("g")</code>
<code>mutate(m = mean(x), .by = g)</code>	<code>groupby("g")["x"].transform("mean")</code>	<code>pl.col("x").mean().over("g")</code>
<code>across(where(is.numeric), f)</code>	<code>select_dtypes("number").apply(f)</code>	<code>cs.numeric().f()</code>
<code>rename(new = old)</code>	<code>rename(columns={"old": "new"})</code>	<code>rename({"old": "new"})</code>
<code>left_join(y, by = "k")</code>	<code>merge(y, on="k", how="left")</code>	<code>join(y, on="k", how="left")</code>
<code>pivot_longer() / pivot_wider()</code>	<code>melt() / pivot()</code>	<code>unpivot() / pivot()</code>

Takeaways

- A pandas Series is a typed array plus an index, and the index is what makes pandas different from R - lookup is by label, arithmetic and assignment align on labels, and group keys end up in the index. `.loc` and `.iloc` exist because `[]` cannot tell labels from positions.
- pandas dtypes are messier than R's - mixed data becomes `object`, and missing values are `NaN`, `None`, `NaT`, or `pd.NA` depending on the dtype, with `NaN` silently turning integer columns into floats. Use `isna()`, never `==`, and remember that aggregations skip missing values by default.
- Python has no data masking, so pandas refers to columns with strings, callables, or `pd.col()` and chains methods instead of piping. DataFrames are mutable, so aliases matter.
- polars drops the index and adds a strict schema, `null` for every dtype, expressions evaluated inside contexts, and a lazy mode with a query optimizer - much of which will look familiar again when we get to SQL and DuckDB.